

TP 310

En vous inspirant du guide d'installation de postgresql, vous réaliserez dans votre machine virtuelle linux les points suivants :

- Installer le service postgresql
 - `sudo apt install -y postgresql postgresql-contrib`
 - Vérifiez que le processus est bien actif
 - `systemctl status postgresql`
 - Modifiez le mot de passe par défaut de l'administrateur
 - Dans `psql`:

```
\password postgres
```

19

- Testez de vous connectez

C'est bonne

```
postgres=# \password postgres
Enter new password for user "postgres":
Enter it again:
postgres=#
postgres=# \q
postgres@rt-mv:~$ psql
psql (14.18 (Ubuntu 14.18-0ubuntu0.22.04.1))
Type "help" for help.

postgres=# \q
postgres@rt-mv:~$ psql
psql (14.18 (Ubuntu 14.18-0ubuntu0.22.04.1))
Type "help" for help.

postgres=#
```

- Créez un utilisateur *invite* (Utilisez la commande `\du` de psql pour afficher la liste des utilisateurs)

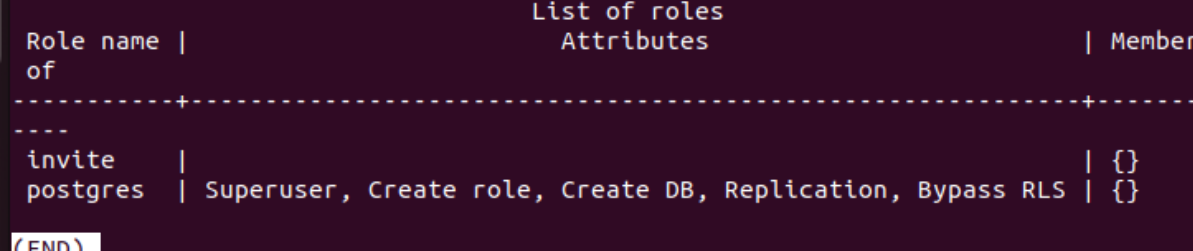
```

List of roles
Role name | Attributes | Member of
-----+-----+-----
postgres | Superuser, Create role, Create DB, Replication, Bypass RLS | {}
(END)

```

Commande CREATE USER invite WITH PASSWORD 'progr00';

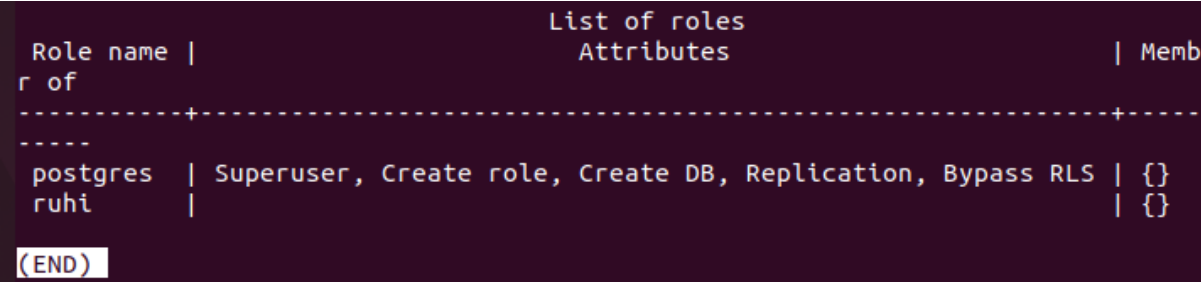
Pour le voir \du



```
Role name | List of roles | Member
of        | Attributes    |
-----+-----+-----
invite   | Superuser, Create role, Create DB, Replication, Bypass RLS | {}
postgres| {}
```

(END)

Comme on peut voir il y pas de user pour l'instant on utilise la commande:



```
Role name | List of roles | Memb
r of      | Attributes    |
-----+-----+-----
postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS | {}
ruhi      | {}
```

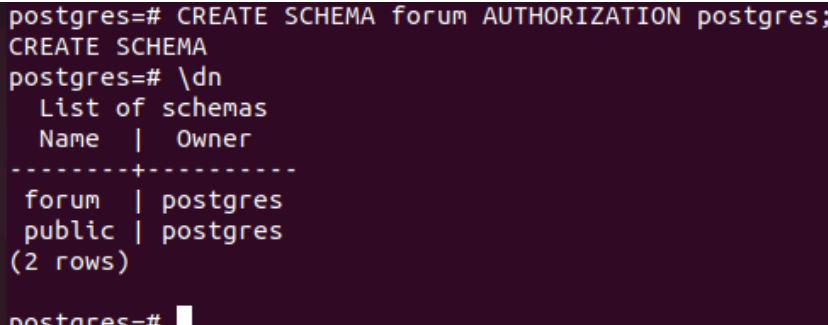
(END)

Ctrl droite

- Créez un schéma *forum* (Utilisez la commande \dn de psql pour afficher la liste des schémas)

La commande:

- CREATE SCHEMA forum AUTHORIZATION postgres;
- \dn



```
postgres=# CREATE SCHEMA forum AUTHORIZATION postgres;
CREATE SCHEMA
postgres=# \dn
List of schemas
Name | Owner
-----+-----
forum | postgres
public | postgres
(2 rows)

postgres=#
```

- Créez dans ce schéma les deux tables *personne* et *message*

La commande

```
CREATE TABLE forum.personne (  
    id_personne    SERIAL PRIMARY KEY,  
    nom            VARCHAR(50) NOT NULL,  
    prenom         VARCHAR(50) NOT NULL,  
    pseudo         VARCHAR(50) NOT NULL,  
    date_naissance DATE  
);  
  
-- Table message  
CREATE TABLE forum.message (  
    id_message     SERIAL PRIMARY KEY,  
    id_personne    INTEGER NOT NULL REFERENCES  
forum.personne(id_personne),  
    contenu        VARCHAR  
);
```

```
date_naissance DATE  
);  
-- Table message  
CREATE TABLE forum.message (  
    id_message     SERIAL PRIMARY KEY,  
    id_personne    INTEGER NOT NULL REFERENCES forum.personne(id_personne),  
    contenu        VARCHAR  
);  
CREATE TABLE  
CREATE TABLE  
postgres=# \dt forum.*  
List of relations  
Schema | Name      | Type | Owner  
-----+-----+-----+-----  
forum  | message  | table | postgres  
forum  | personne | table | postgres  
(2 rows)  
postgres=#
```

- à partir de ce point, vérifiez avec la commande *psql* adéquate les modifications que vous apportez à la base de données

La commande:

- *\du* ⇒ Vérifier les rôles (utilisateurs)
- *\dn* ⇒ Montre la liste des schémas
- *\dt forum.** ⇒ Montre les tables créées dans forum (après CREATE TABLE).
- *\dt forum.** ⇒ Vérifie que les tables sont listées.
-
-

```
postgres=# \dt forum.*
          List of relations
 Schema |   Name   | Type | Owner
-----+-----+-----+-----
 forum | message | table | postgres
 forum | personne | table | postgres
(2 rows)

postgres=#
```

- Insérez quelques données dans les 2 tables

Les commande:

```
INSERT INTO forum.personne (nom, prenom, pseudo, date_naissance)
VALUES
```

```
('dupont', 'jean', 'jdupont', '2000-05-14'),
```

```
('martin', 'claire', 'cmartin', '1999-12-01'),
```

```
('bernard', 'luc', 'lbernard', '1998-07-22');
```

And then

```
INSERT INTO forum.message (id_personne, contenu) VALUES
```

```
(1, 'Bonjour à tous !'),
```

```
(2, 'Hey to you.'),
```

```
(3, 'Who likes Apple?');
```

```
postgres=# INSERT INTO forum.personne (nom, prenom, pseudo, date_naissance) VALUES
('heyThere', 'idk', 'player', '2006-05-14'),
('idc', 'idk', 'player', '2005-12-01'),
('sherzad', 'ruhi', 'player', '2004-07-22');
INSERT 0 3
postgres=# INSERT INTO forum.message (id_personne, contenu) VALUES
(1, 'Bonjour à tous !'),
(2, 'hey to you.'),
(3, 'who like's applle');
```

- Permettre à l'utilisateur *invite* d'accéder au schéma *forum*

La commande

- GRANT USAGE ON SCHEMA forum TO Ruhi;

On peut change Ruhi contre d'autres nom

- Permettre à l'utilisateur *invite* d'ajouter des messages

La commande

```
GRANT USAGE, SELECT, UPDATE ON SEQUENCE
forum.message_id_message_seq TO invite;
```

Vérification des droits

\z forum.*

Schema	Name	Type	Access privileges	Column privileges
Access privileges	Policies			
forum	message	table	postgres=arwdBxt/postgres+	
			ruhi=a/postgres	
forum	message_id_message_seq	sequence	postgres=rwU/postgres	+
			ruhi=rwU/postgres	
forum	personne	table		
forum	personne_id_personne_seq	sequence		
(4 rows)				
(END)				

- Vérifiez le bon fonctionnement de l'ajout
- Vérifiez que l'utilisateur *invite* ne peut pas supprimer de message
- Vérifiez que l'utilisateur *invite* ne peut rien faire sur la table *personne*

```
GRANT USAGE ON SCHEMA forum TO Ruhi;
GRANT INSERT ON forum.message TO Ruhi;
GRANT USAGE, SELECT, UPDATE ON SEQUENCE
forum.message_id_message_seq TO Ruhi;
```

```
postgres=# GRANT USAGE ON SCHEMA forum TO Ruhi;
GRANT INSERT ON forum.message TO Ruhi;
GRANT USAGE, SELECT, UPDATE ON SEQUENCE forum.message_id_message_seq TO Ruhi;
GRANT
GRANT
GRANT
postgres=#
```

```
postgres=# \du
postgres=# psql -U invite -d postgres -h 127.0.0.1 -W
postgres=#
postgres=# psql -U Ruhi -d postgres -h 127.0.0.1 -W
postgres=# psql -U Ruhi2 -d postgres -h 127.0.0.1 -W
postgres=# INSERT INTO forum.message (id_personne, contenu) VALUES (1, 'Message
par Ruhi2');
ERROR: syntax error at or near "psql"
LINE 1: psql -U invite -d postgres -h 127.0.0.1 -W
        ^
postgres=# DELETE FROM forum.message WHERE id_message = 1; -- doit échouer
DELETE 0
postgres=# SELECT * FROM forum.personne; -- doit échouer
 id_personne |  nom   | prenom | pseudo | date_naissance
-----+-----+-----+-----+-----
          1 | heyThere | idk    | player | 2006-05-14
          2 | idc     | idk    | player | 2005-12-01
          3 | sherzad | ruhi   | player | 2004-07-22
(3 rows)
postgres=#
```

Faites un dump du schéma *forum*

- Un dump complet

Commande

- \q
- pg_dump -U postgres -n forum -d postgres > forum_full.sql

- Un dump avec que la structure

Commande

- pg_dump -U postgres -n forum --schema-only -d postgres > forum_structure.sql
- Un dump avec uniquement les données

Commande

- pg_dump -U postgres -n forum --data-only -d postgres > forum_data.sql

Puis on execute

ls -lh forum_*.sql

Pour voir

- forum_full.sql
- forum_structure.sql
- forum_data.sql

```
postgres@rt-mv:~$ ls -lh forum_*.sql
-rw-rw-r-- 1 postgres postgres 1,3K sept.  5 16:33 forum_data.sql
-rw-rw-r-- 1 postgres postgres 4,4K sept.  5 16:33 forum_full.sql
-rw-rw-r-- 1 postgres postgres 3,6K sept.  5 16:33 forum_structure.sql
postgres@rt-mv:~$
```

- Créez un nouveau schéma *backup*

Il faut exécuter cette commande dans *psql*

Commande ⇒ *psql -U postgres -d postgres*

Puis

Commande pour créer backup ⇒ *CREATE SCHEMA backup;*

```
postgres=# CREATE SCHEMA backup;
CREATE SCHEMA
postgres=# \dn
List of schemas
Name | Owner
-----+-----
backup | postgres
forum | postgres
public | postgres
(3 rows)

postgres=# ^C
postgres=#
```

Dans le schéma *backup* :

- Restorez dans un premier temps la structure du schéma *forum*

La commande ⇒ *sed -E "s/\bforum\./backup./g; s/SCHEMA forum/SCHEMA backup/g" forum_structure.sql > backup_structure.sql*

psql -U postgres -d postgres -f backup_structure.sql

- Ajoutez ensuite les données restaurées

11) Schéma fullrestore

```
CREATE SCHEMA fullrestore;
```

```
sed -E "s/\bforum\./fullrestore./g" forum_full.sql > fullrestore.sql
```

```
psql -U postgres -d postgres -f fullrestore.sql
```

12) Annexes

```
\l : bases
```

```
\dn : schémas
```

```
\dt schema.* : tables
```

```
\du : rôles
```

```
\z schema.* : droits
```

Pré-requis & conventions

- Distribution courante (Ubuntu/Debian) avec **systemd**.
- Compte super-utilisateur PostgreSQL local : **postgres**.
- Client en ligne de commande : **psql**.
- Schéma demandé : **forum**.
- Nouvel utilisateur : **invite**.

Astuce psql :

- **\?** : aide générale psql
- **\h** : aide SQL (ex. **\h create table**)
- **\l** : lister bases
- **\du** : lister rôles/utilisateurs
- **\dn** : lister schémas
- **\dt** : lister tables (ex. **\dt forum.***)
- **\d nom_table** : description table
- **\z** : droits d'accès

1) [Installer PostgreSQL](#)

(Ubuntu/Debian)

```
sudo apt update
```

```
sudo apt install postgresql postgresql-contrib
```

Explications :

- **apt install postgresql** : installe le serveur PostgreSQL (service + binaires **psql**, **pg_dump**, etc.).
- **postgresql-contrib** : modules supplémentaires utiles (extensions).

2) Vérifier que le service tourne

```
sudo systemctl status postgresql
```

Explications :

- **systemctl status** : affiche l'état du service systemd.
- Vous devez voir "active (running)".
Au besoin :

```
sudo systemctl enable postgresql # démarrage automatique au boot
sudo systemctl start postgresql  # démarrer maintenant
```

3) Changer le mot de passe de l'administrateur (**postgres**)

Passez sur le compte système **postgres** et lancez **psql** :

```
sudo -iu postgres
psql
```

Dans **psql**, changez le mot de passe du rôle super-utilisateur **postgres** :

```
ALTER ROLE postgres WITH PASSWORD 'MotDePasseFort!';
```

Explications :

- **sudo -iu postgres** : ouvre un shell en tant qu'utilisateur système **postgres**.
- **psql** : ouvre le client connecté au cluster local, BD par défaut.
- **ALTER ROLE ... PASSWORD** : définit le mot de passe du rôle (utilisateur) PostgreSQL.

Quittez **psql** puis le shell :

```
\q
exit
```

4) Créer (ou choisir) une base de données de travail

Ici j'utilise **rt3** :

```
sudo -iu postgres createdb rt3
```

Explications :

- **createdb rt3** : crée une base nommée **rt3** dans le cluster.
-

5) Tester la connexion

Depuis votre compte normal Linux :

```
psql -U postgres -d rt3
```

Explications :

- `-U postgres` : se connecter avec le rôle `postgres`
 - `-d rt3` : base cible
-

6) Créer l'utilisateur `invite`

Dans `psql` (connecté à `rt3` en tant que `postgres`) :

```
CREATE ROLE invite LOGIN PASSWORD 'invite123';
```

Vérifier :

```
\du
```

Explications :

- `CREATE ROLE ... LOGIN` : un rôle avec droit de connexion (équivalent à "utilisateur").
 - `\du` : liste les rôles, leurs attributs (Superuser, Create DB, etc.).
-

7) Créer le schéma `forum`

Toujours dans `psql` :

```
CREATE SCHEMA forum AUTHORIZATION postgres;
```

Vérifier :

```
\dn
```

Explications :

- `CREATE SCHEMA forum` : espace logique pour regrouper objets (tables, vues...).

- **AUTHORIZATION postgres** : propriétaire du schéma.
-

8) Créer les tables **forum.personne** et **forum.message**

Le sujet donne des **integer**; je rajoute une clé étrangère cohérente (non exigée mais bonne pratique).

-- Table personne

```
CREATE TABLE forum.personne (  
  id_personne integer PRIMARY KEY,  
  nom          varchar(50) NOT NULL,  
  prenom       varchar(50) NOT NULL,  
  pseudo       varchar(50) NOT NULL,  
  date_naissance date  
);
```

-- Table message

```
CREATE TABLE forum.message (  
  id_message integer PRIMARY KEY,  
  id_personne integer NOT NULL,  
  contenu     varchar,  
  CONSTRAINT fk_message_personne  
    FOREIGN KEY (id_personne)  
    REFERENCES forum.personne(id_personne)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT  
);
```

Vérifier :

```
\dt forum.*  
\d forum.personne  
\d forum.message
```

Explications (principales options) :

- **PRIMARY KEY** : unicité + index.
- **NOT NULL** : valeur obligatoire.
- **FOREIGN KEY ... REFERENCES** : intégrité référentielle.
 - **ON UPDATE CASCADE** : si la clé primaire change, la FK suit (rare mais correct).

- **ON DELETE RESTRICT** : empêche suppression d'une personne si messages associés.

Remarque : Si vous préférez l'auto-incrément, utilisez **GENERATED BY DEFAULT AS IDENTITY** sur les id (PostgreSQL ≥10).

9) Insérer quelques données

```
INSERT INTO forum.personne (id_personne, nom, prenom, pseudo,
date_naissance) VALUES
(1, 'dupont', 'alice', 'ali', '1999-01-12'),
(2, 'martin', 'bruno', 'bru', '2000-05-03');
```

```
INSERT INTO forum.message (id_message, id_personne, contenu)
VALUES
(101, 1, 'bonjour à tous'),
(102, 1, 'qui vient au cours ?'),
(103, 2, 'je viens !');
```

Vérifier :

```
TABLE forum.personne;
TABLE forum.message;
```

10) Droits : donner accès au schéma **forum** à **invite**

```
-- 1) Autoriser l'entrée dans le schéma (vu comme un "répertoire")
GRANT USAGE ON SCHEMA forum TO invite;
```

```
-- 2) Donner le droit d'ajouter des messages (INSERT) mais pas de
supprimer
GRANT INSERT ON forum.message TO invite;
```

```
-- (Optionnel) Autoriser SELECT pour que 'invite' voie les messages
existants
GRANT SELECT ON forum.message TO invite;
```

```
-- S'assurer que 'invite' n'a aucun droit sur 'personne'
REVOKE ALL ON forum.personne FROM invite;
```

Vérifier les privilèges :

\z forum.*

Explications clés :

- **GRANT USAGE ON SCHEMA** : nécessaire pour accéder aux objets du schéma.
 - **GRANT INSERT ON table** : autorise uniquement les insertions.
 - **REVOKE ALL** : retire toute permission (SELECT/INSERT/UPDATE/DELETE/TRIGGER/REFERENCES).
-

11) Tester depuis l'utilisateur **invite**

Quittez **psql** (\q). Puis :

```
psql -U invite -d rt3
```

Test **INSERT** (doit fonctionner) :

```
INSERT INTO forum.message (id_message, id_personne, contenu)
VALUES (104, 2, 'message ajouté par invite');
```

Test **DELETE** (doit échouer) :

```
DELETE FROM forum.message WHERE id_message = 101;
-- => ERROR: permission denied for table message
```

Test accès à **personne** (doit être interdit si vous n'avez pas donné SELECT) :

```
SELECT * FROM forum.personne;
-- => ERROR: permission denied for table personne
```

12) DUMP du schéma **forum**

Nous allons produire 3 fichiers dans le dossier courant.

12.1 Dump complet (structure + données)

```
pg_dump -U postgres -d rt3 -n forum -F c -f forum_full.dump
```

Explications paramètres :

- `-U postgres` : rôle utilisé pour dumper.
- `-d rt3` : base source.
- `-n forum` : uniquement le schéma `forum`.
- `-F c` : format custom (binaire) — idéal pour `pg_restore`.
- `-f forum_full.dump` : fichier de sortie.

12.2 Dump structure seule

```
pg_dump -U postgres -d rt3 -n forum -F c -s -f forum_structure.dump
```

- `-s / --schema-only` : sans les données.

12.3 Dump données seules

```
pg_dump -U postgres -d rt3 -n forum -F c -a -f forum_data.dump
```

- `-a / --data-only` : sans la structure, uniquement les `INSERT`.

Astuce : vous pouvez aussi produire des dumps texte (SQL lisible) avec `-F p` et rediriger dans `.sql`.

13) Créer un nouveau schéma `backup`

Reconnectez-vous en `postgres` si besoin et :

```
psql -U postgres -d rt3
```

```
CREATE SCHEMA backup AUTHORIZATION postgres;  
\dn
```

14) Restaurer d'abord la structure de `forum` dans `backup`

`pg_restore` ne "renomme" pas de schéma. On a deux options pédagogiques :

1. Méthode simple et 100% SQL (recommandée en TP) :
recréer la structure avec `LIKE` (copie)
2. Méthode `pg_restore` + réécriture (sed) du schéma dans
un dump texte

14.A — Méthode simple (SQL pur, très propre en TP)

```
-- Recréer les tables dans 'backup' à partir de la structure de 'forum'  
CREATE TABLE backup.personne (LIKE forum.personne INCLUDING ALL);  
CREATE TABLE backup.message (LIKE forum.message INCLUDING ALL);
```

LIKE ... INCLUDING ALL copie colonnes, contraintes, index, identités, etc.

Vérifiez :

```
\dt backup.*  
\d backup.personne  
\d backup.message
```

14.B — (Option alternative avancée) Méthode pg_restore + remap de schéma

Si vous tenez à **utiliser les dumps structurels** strictement :

1. Générer un **dump texte** de la structure :

```
pg_dump -U postgres -d rt3 -n forum -F p -s -f forum_structure.sql
```

2. Remplacer **forum** par **backup** dans le fichier :

```
sed -e 's/SET search_path = forum/SET search_path = backup/g' \  
-e 's/ forum\./ backup\./g' \  
forum_structure.sql > forum_structure_for_backup.sql
```

3. Charger dans la base :

```
psql -U postgres -d rt3 -f forum_structure_for_backup.sql
```

Cette voie illustre bien la **restauration structurelle** dans un autre schéma.

15) Ajouter ensuite les données restaurées dans **backup**

15.A — Méthode simple (SQL pur)

```
INSERT INTO backup.personne SELECT * FROM forum.personne;  
INSERT INTO backup.message SELECT * FROM forum.message;
```

Vérifier :

```
TABLE backup.personne;  
TABLE backup.message;
```

15.B — (Alternative) Charger via dump data-only

Si vous voulez absolument réutiliser `forum_data.dump` :

1. Le convertir en SQL texte :

```
pg_restore -U postgres -d rt3 -n forum -a -F c -f forum_data.sql  
forum_data.dump
```

2. Adapter les noms de schéma :

```
sed 's/INSERT INTO forum\./INSERT INTO backup./g' forum_data.sql >  
forum_data_for_backup.sql
```

3. Charger :

```
psql -U postgres -d rt3 -f forum_data_for_backup.sql
```

16) Restaurer le schéma `forum` (dump complet) dans un schéma `fullrestore`

On crée `fullrestore`, puis on importe TOUT du dump complet en le remappant vers `fullrestore`. Même logique : `pg_restore` ne renomme pas de schéma → on passe par un `dump texte` puis remplacement.

```
psql -U postgres -d rt3 -c "CREATE SCHEMA fullrestore AUTHORIZATION  
postgres;"
```

1. Convertir le dump binaire `complet` en SQL texte :

```
pg_restore -U postgres -d rt3 -n forum -F c -f forum_full.sql  
forum_full.dump
```

2. Remapper vers `fullrestore` :

```
sed -e 's/SET search_path = forum/SET search_path = fullrestore/g' \  
-e 's/ forum\./ fullrestore./g' \  
forum_full.dump
```

```
forum_full.sql > forum_full_for_fullrestore.sql
```

3. Charger :

```
psql -U postgres -d rt3 -f forum_full_for_fullrestore.sql
```

Vérifier :

```
psql -U postgres -d rt3 -c "\dt fullrestore.*"
```

```
psql -U postgres -d rt3 -c "TABLE fullrestore.personne;"
```

```
psql -U postgres -d rt3 -c "TABLE fullrestore.message;"
```

Points de contrôle psql recommandés (à insérer dans votre compte-rendu)

- Utilisateurs : `\du`
- Schémas : `\dn`
- Tables par schéma : `\dt forum.*`, `\dt backup.*`, `\dt fullrestore.*`
- Descriptions : `\d forum.personne`, `\d forum.message`
- Droits : `\z forum.*`
- Tests de permissions (connecté en `invite`) :
 - `INSERT` OK sur `forum.message`
 - `DELETE` refusé sur `forum.message`
 - `SELECT` refusé (ou autorisé si vous l'avez donné) sur `forum.personne`

Annexes — explications de paramètres utiles

- `psql -U user -d db -h host -p port`
 - `-U` : rôle PostgreSQL
 - `-d` : base cible
 - `-h` : hôte (défaut : socket local)
 - `-p` : port (défaut : 5432)
- `pg_dump` : export
 - `-n forum` : ne dumper que le schéma `forum`
 - `-F c|p` : format *custom* (c) binaire / *plain* (p) texte SQL
 - `-s` : *schema-only* (structure)

- `-a` : data-only (données)
 - `-f` : fichier de sortie
- `pg_restore` : import de dumps au format *custom* (`-F c`)
 - `-n forum` : ne restaurer que ce schéma depuis le dump (ne renomme pas)
 - `-f fichier.sql` : écrire du SQL *texte* au lieu d'appliquer directement
- `GRANT/REVOKE`
 - `GRANT USAGE ON SCHEMA sch` : droit d'entrer dans le schéma
 - `GRANT SELECT/INSERT/UPDATE/DELETE ON table` : droits par table
 - `\z` : affiche les ACL (listes de contrôle d'accès)